VHDL – Lab 5

Component instantiation – Serial UART

Address:
Mor Kerstins väg 27
511 53 Kinna
Sweden

Tel:
+46-703-748429

Registration no:
559037-6603

E-mail:
kent@elekteknik.se

Revision history

Date	Author	Description
2023-04-01	Kent Abrahamsson	First edition

Address:
Mor Kerstins väg 27
511 53 Kinna
Sweden

Tel:
+46-703-748429

Registration no:
559037-6603

E-mail:
kent@elekteknik.se

1 Table of Contents

1 TABLE OF CONTENTS 3

2 GENERAL 4

2.1 Background 4

2.2 Intended outcome 4

2.3 Checkpoints 4

3 LAB INSTRUCTIONS 5

3.1 General 5

3.2 Serial UART design / interface description 7

3.2.1 Receive data interface 7

3.2.2 Transmit data interface 7

Address:
Mor Kerstins väg 27
511 53 Kinna
Sweden

Tel:
+46-703-748429

Registration no:
559037-6603

E-mail:
kent@elekteknik.se

2 General

2.1 Background

If you wish to reuse an already finished VHDL design in a new project or if you wish to use several copies of one VHDL design in a single project we need a way to reuse an old design without copy-pasting code. This lab focuses on the very important concept of hierarchical/structural VHDL-designs. As explained in the lectures, VHDL offers a nice way to reuse designs that you have already designed and tested, or use designs that someone else has written and tested.

The VHDL-designs, more commonly referred to as “VHDL components” can be reused as black box, where the only thing you need to care about is the interface of the component (i.e. the entity), and of course how the interface signals shall be handled to make the component work as specified.

Components can be delivered as a normal set of VHDL files, but if you buy components from a supplier they may also deliver the component as a netlist file, i.e. already synthesized component. If they deliver the component as a synthesized netlist, this is done to protect their intellectual property or “IP”. Components may because of this also be called IP-blocks as in “Intellectual Property” blocks.

2.2 Intended outcome

In this lab the goal is to learn how to reuse two already finished designs in a new top level design. You shall also create a new component which interfaces to one of the existing components.

The idea is to create a fully structural top level file where the sub components are instantiated.

2.3 Checkpoints

In order to Pass the lab assignment the student shall be able to show that the following tasks have been fulfilled.

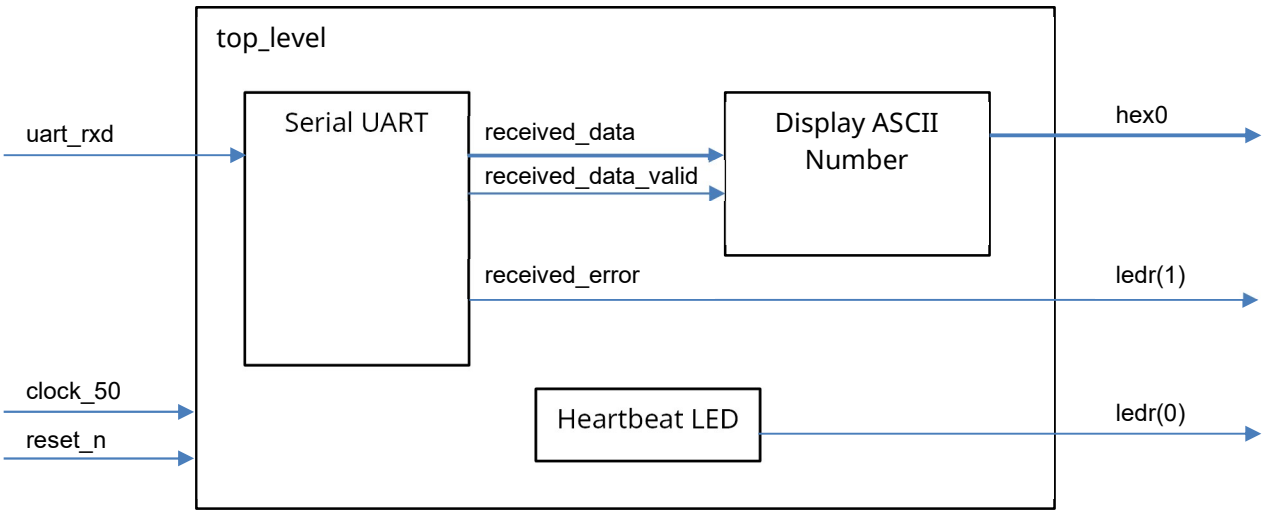
- Properly written VHDL implementation. Use proper indentation, spaces are cheap! Add comments when you feel it’s suitable.
- The top level shall be a pure structural design, i.e. only containing component instantiations and internal signals in the architecture, no processes or logic in the top level file.
- A key push button shall be used as reset in the design.
- The Serial UART component generics are properly configured in “generic map”.
- A ModelSim simulation shall be performed where two different ASCII characters is simulated.
- The RTL (Netlist) viewer shall be viewed but not analyzed in detail.
- Design shall also be shown to work in hardware.

Tip: Check out the ModelSim tips in the end of this document.

3 Lab instructions

3.1 General

The top level design file shall be design as shown in the block diagram below.



Suggested workflow:

1. Download the Serial UART component from the course homepage. Study the Serial UART design/interface description in section 3.2.

   Create a new top_level design file and instantiate the Serial UART component, connect the signals as shown in the block diagram above, configure the generic settings to use:

   - 9600 bps
   - No parity

   Since the uart_rxd (Serial receive data pin) on the FPGA is totally asynchronous to the internal 50 MHz clock, study the code in the Serial UART component to ensure that this signal is double synchronized.

   Leave any unused outputs on the Serial UART component open, and make sure unused inputs are tied to an inactive state.

2. Instantiate the LED blink component you designed in lab 3 (called “Heartbeat LED” in block diagram above), connect the LED output to LEDR#0.

3. Create a new “Display ASCII Number” VHDL component, which interfaces the Serial UART component through its entity, and also connects to the seven segment display on hex0 output.

   The “Display ASCII Number” component shall receive the byte of data from the Serial UART and output the ASCII characters for numbers 0-9 when these are received on the serial interface, all unknown characters shall give a “-“ on the seven segment display.

   Instantiate this new component in the top level as well and connect it to the signals required.

4. Ensure that all files are properly added to the Quartus project and that all files are stored in the lab project folder.
5. Run the design through synthesis to ensure that no syntax errors exist.
6. Simulate the design in ModelSim.

   Simulating this design is more challenging than previous labs. Ensure as always that all inputs are defined to an idle state (uart_rxd should be hign in idle) and reset is active for a couple (>5 clock cycles). Release reset and run for another couple of clock cycles before changing any other inputs. To check the heartbeat LED, it could be wise to temporarily change the delay in the component to a much lower value and verify that part of the design (just as you did in lab3). DO NOT forget to change back to the correct delay after this has been verified.

   To verify the serial data, make sure the simulation started with uart_rxd input high, representing an idle serial line. Then run the simulator for 1ms to make sure everything is in "idle", no undefined signals etc.

   Then force uart_rxd low, and run the simulator for say 5 bit times, where the bit time length is 104us at 9600bps. This will simulate a half received byte. Then force rx_in back high, and run for say 1ms more. This should fully cover the reception of the remaining bits and also show you activity on the received_data_valid signal, followed by output on the seven segment vector (hex0). To simulate the reception of a certain ASCII-code, it will require that you run the simulator one bit at a time and change the uart_rdx line accordingly. This can be a bit tricky but not difficult at all, start with start bit, followed by LSB and so on…
7. When you are certain that the design responds to the inputs as you expect, pin lock the pins and run a complete place and route and download to the development board. If you use the DE1-SoC board you will need the custom made DE1 Expansion board, HW revision 2 and use the USB to serial component (CP2102) located on that board. Or you will need another USB to serial cable to hook up directly to the GPIO header. Look at the schematics for the DE1 expansion board (if used) and the user manual to pin lock the uart_rxd pin to the correct position on the GPIO header.
8. Start PuTTY and configure 9600bps, 8N1 (8 data bits, No parity, 1 stop bit). Connect the serial cable between the PC COM-port and the development board. Type some characters and numbers on the PC keyboard and verify that the design properly receives the serial data stream and decodes the ASCII-character 0-9 correct. Also verify the heartbeat LED.
9. Check the resource utilization of the finished design, not number of logical elements and registers.
10. Finally – think about how this design is truly parallel in the FPGA compared to how a microcontroller would solve the same tasks.

Extra:

- Connect the received_data_valid and received_data vector signals to the transmit_data_valid and the transmit_data signals on the Serial UART component. Add tx_out signal in the top_level entity and pin map that signal to the right pin on the FPGA to test sending the data out that was just received.

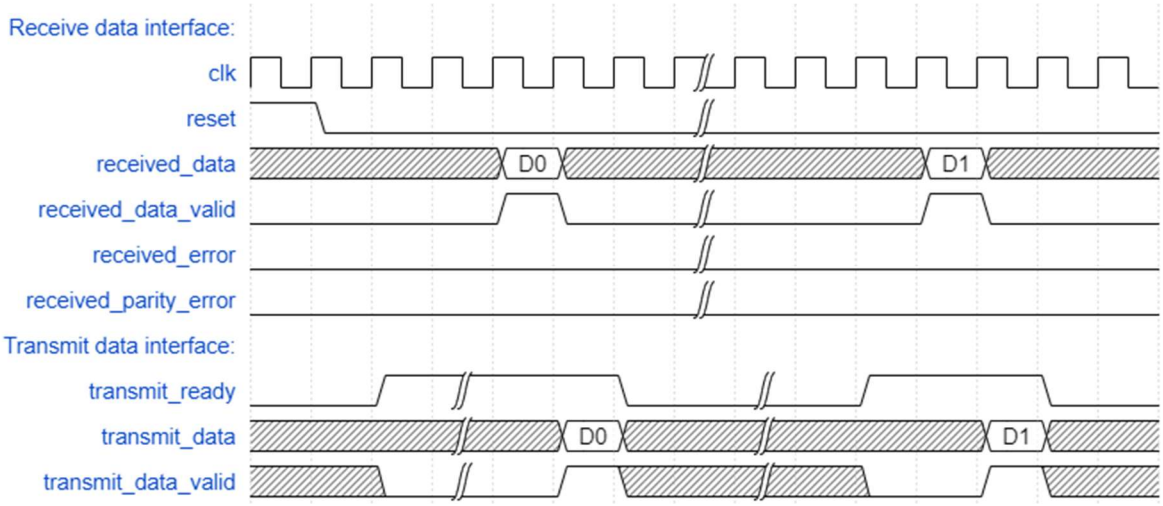
- Compare the synthesis result with the previous design discuss differences (RTL viewer inside the Serial UART component, and look at Synthesis Flow report numbers).

3.2 Serial UART design / interface description

The Serial UART component is designed to send and/or receive data through a serial interface. It provides a simple user interface for sending and/or receiving one byte of data at a time.

The serial link speed is set up using generics where the clock period and requested link speed is set up. Generics are also used to set up parity setting and the reset input active level.

All inputs and outputs in the user interface is synchronous to the clk signal.



User interface timing diagram examples are shown below and is further explained in the following chapters.

Note that the rx and tx serial signals are not shown above, only the user interface signals.

3.2.1 Receive data interface

The receive interface provides the received byte of data on the received_data output vector.

The receive_data_valid signal is pulsed high one cycle when a new byte of data has been received successfully. The received_error signal is set high if an error in the receive process is detected, e.g. a missing stop bit in the serial data. If parity error is detected the received_parity_error will be set high, the parity error check is disabled if the generic settings are set to “no parity”.

3.2.2 Transmit data interface

To transmit one byte of data, the byte that shall be transmitted shall be presented on the transmit data vector and the transmit_data_valid signal shall be set high. The byte provided on the transmit_data signal will be sampled and sent by the Serial UART component if the transmit_ready output is set high simultaneously as the transmit_data_valid signal is high.

Both transmit_data and transmit_data_valid input signals towards the Serial UART component are ignored when the transmit_ready output from the Serial UART component is set low meaning that these can be set to any value during this time.